Employee Document & Request Management Portal

Capstone Project | SharePoint Online Developer Training | SPFx, React, PnPjs, Microsoft Graph, Power Automate

1. Problem Statement

In many organizations, employees raise leave, equipment, travel and HR-letter requests through email threads, chat messages and shared drives. This creates five recurring problems:

- Requests get lost and nobody can see their current status.
- Employee documents (ID proofs, offer letters, certificates) are scattered across personal drives.
- Approvals are manual, slow and hard to audit.
- Access control is weak, so sensitive HR content can be seen by the wrong people.
- Managers and HR have no consolidated view of what is pending.

Goal: build one secure portal on SharePoint Online where employees raise requests and manage documents, managers approve from Outlook or Teams, and HR administers everything, with access controlled by role.

Objectives

- Provide a custom React dashboard showing request counts and recent activity.
- Store data in SharePoint Lists and Libraries with a clean data model.
- Perform create, read, update and delete operations with PnPjs.
- Show user profile data through Microsoft Graph.
- Offer search and filter across requests and documents.
- Automate approvals with Power Automate.
- Enforce role-based permissions.
- Add a global banner and status badges with SPFx extensions.
- Package and deploy the solution (.sppkg) to SharePoint Online.

2. Introduction

The Employee Document & Request Management Portal (EDRMP) is a end-to-end SharePoint Online solution that combines the platform's no-code strengths (lists, libraries, permissions, Power Automate) with pro-code customisation (SharePoint Framework, React, PnPjs, Microsoft Graph). It exercises every module of the training: SharePoint fundamentals, SPFx, APIs and Graph, advanced extensions, and deployment.

2.1 Solution overview

Requirement	Solution component
Custom dashboard	SPFx React web part `EmployeeDashboard`
Lists and libraries	`Employee Requests`, `Employee Directory`, `Employee Documents`
CRUD operations	PnPjs v3 in a service layer (`RequestService`)

Requirement	Solution component
Graph integration	MSGraphClientV3: profile, manager, photo
Search and filter	OData $filter plus SharePoint Search
Approval workflow	Power Automate flow on Employee Requests
Role-based permissions	SharePoint groups, unique permissions, item-level security
Custom extension	Application Customizer (banner) and Field Customizer (status badge)
Deployment	.sppkg in the tenant App Catalog

2.2 Prerequisites

- Microsoft 365 tenant with SharePoint Online and a Power Automate licence.
- Site collection owner rights on the portal site and App Catalog access (SharePoint admin for API approvals).
- Node.js v22 LTS, Visual Studio Code, yo and @microsoft/generator-sharepoint.
- Browser DevTools console for running the SharePoint REST scripts.

2.3 Data model

Object	Type	Columns (internal name: type)
Employee Requests	List	Title: text, RequestType: choice (Leave, Equipment, Travel, HR Letter, Other), Priority: choice (Low, Medium, High), Status: choice (Pending, Approved, Rejected; default Pending), Description: multi-line, RequiredBy: date, ApproverComments: multi-line
Employee Directory	List	Title: text, Department: text, Experience: number, IsActive: yes/no (same shape as the REST Demo Employees list)
Employee Documents	Library	DocumentType: choice (ID Proof, Offer Letter, Certificate, Payslip), Employee: person, ExpiryDate: date

3. Cases and Solutions

Case 1: Environment setup and project scaffolding

Scenario: the team needs a working SPFx development environment and a solution skeleton.

Solution steps

1. Verify tools and install the generator:

```
node -v
npm install -g yo @microsoft/generator-sharepoint
mkdir employee-portal
```

```
cd employee-portal
yo @microsoft/sharepoint
```

2. Answer the prompts: solution name `employee-portal`, SharePoint Online only, current folder, tenant-wide deployment **Yes**, component type **WebPart**, name `EmployeeDashboard`, framework **React**.
3. Install libraries and trust the dev certificate:

```
npm install @pnp/sp @pnp/graph @pnp/logging @pnp/queryable --save
heft trust-dev-cert
heft start
```

4. Open the hosted workbench at `https://<tenant>.sharepoint.com/_layouts/workbench.aspx` and add the web part.

Expected result: the default web part renders in the workbench with live reload.

Case 2: SharePoint lists and libraries

Scenario: create the data layer before writing any UI code.

Solution steps (run once as site owner, from the browser console on any page of the portal site):

```
// Run from the browser console on any page of your portal site
var site = _spPageContextInfo.webServerRelativeUrl;
var vhdr = { Accept: 'application/json;odata=verbose', 'Content-Type': 'application/json;odata=verbose' };

async function getDigest() {
  var r = await fetch(`${site}/_api/contextinfo`, { method: 'POST', headers: { Accept: vhdr.Accept } });
  return (await r.json()).d.GetContextWebInformation.FormDigestValue;
}
async function createList(title, template) {   // 100 = list, 101 = document library
  var r = await fetch(`${site}/_api/web/lists`, {
    method: 'POST', headers: { ...vhdr, 'X-RequestDigest': await getDigest() },
    body: JSON.stringify({ __metadata: { type: 'SP.List' }, BaseTemplate: template, Title: title })
  });
  console.log('List', title, r.status);          // 201 = created
}
async function addField(listTitle, schemaXml) {
  var r = await fetch(`${site}/_api/web/lists/getbytitle('${listTitle}')/fields/createfieldasxml`, {
    method: 'POST', headers: { ...vhdr, 'X-RequestDigest': await getDigest() },
    body: JSON.stringify({ parameters: { __metadata: { type: 'SP.XmlSchemaFieldCreationInformation' },
SchemaXml: schemaXml, Options: 8 } })
  });
  console.log('Field', listTitle, r.status);
}

// Employee Requests
await createList('Employee Requests', 100);
await addField('Employee Requests', "<Field Type='Choice' DisplayName='RequestType'
Name='RequestType'><CHOICES><CHOICE>Leave</CHOICE><CHOICE>Equipment</CHOICE><CHOICE>
Travel</CHOICE><CHOICE>HR Letter</CHOICE><CHOICE>Other</CHOICE></CHOICES></Field>");
await addField('Employee Requests', "<Field Type='Choice' DisplayName='Priority'
Name='Priority'><CHOICES><CHOICE>Low</CHOICE><CHOICE>Medium</CHOICE><CHOICE>High</CH
OICE></CHOICES></Field>");
await addField('Employee Requests', "<Field Type='Choice' DisplayName='Status'
Name='Status'><Default>Pending</Default><CHOICES><CHOICE>Pending</CHOICE><CHOICE>Approved</
CHOICE><CHOICE>Rejected</CHOICE></CHOICES></Field>");
```

```
await addField('Employee Requests', "<Field Type='Note' DisplayName='Description' Name='Description'
NumLines='6' RichText='FALSE'/>");
await addField('Employee Requests', "<Field Type='DateTime' DisplayName='RequiredBy' Name='RequiredBy'
Format='DateOnly'/>");
await addField('Employee Requests', "<Field Type='Note' DisplayName='ApproverComments'
Name='ApproverComments' NumLines='4' RichText='FALSE'/>");

// Employee Documents (library)
await createList('Employee Documents', 101);
await addField('Employee Documents', "<Field Type='Choice' DisplayName='DocumentType'
Name='DocumentType'><CHOICES><CHOICE>ID Proof</CHOICE><CHOICE>Offer
Letter</CHOICE><CHOICE>Certificate</CHOICE><CHOICE>Payslip</CHOICE></CHOICES></Field>");
await addField('Employee Documents', "<Field Type='User' DisplayName='Employee' Name='Employee'
List='UserInfo' ShowField='ImnName' UserSelectionMode='0'/>");
await addField('Employee Documents', "<Field Type='DateTime' DisplayName='ExpiryDate' Name='ExpiryDate'
Format='DateOnly'/>");

// Turn on versioning for the library
await fetch(`${site}/_api/web/lists/getbytitle('Employee Documents')`, {
  method: 'POST',
  headers: { ...vhdr, 'X-RequestDigest': await getDigest(), 'IF-MATCH': '*', 'X-HTTP-Method': 'MERGE' },
  body: JSON.stringify({ __metadata: { type: 'SP.List' }, EnableVersioning: true })
});

// Employee Directory
await createList('Employee Directory', 100);
await addField('Employee Directory', "<Field Type='Text' DisplayName='Department' Name='Department'/>");
await addField('Employee Directory', "<Field Type='Number' DisplayName='Experience' Name='Experience'/>");
await addField('Employee Directory', "<Field Type='Boolean' DisplayName='IsActive'
Name='IsActive'><Default>1</Default></Field>");
```

Create `Employee Directory` the same way (`Department`, `Experience`, `IsActive`), or reuse `REST Demo Employees`. Turn on versioning for the library and set the `Status` default to `Pending`.

Verify with the REST call from the console: `/_api/web/lists/getbytitle('Employee Requests')/fields?$filter=Hidden eq false&$select=Title,InternalName,TypeAsString`.

Case 3: Role-based permissions

Scenario: employees must see only their own requests, managers and HR need wider access, and auditors need read-only access.

Group	Permission	Scope
Portal Employees	Contribute	Employee Requests (own items only), Employee Documents (own folder)
Portal Managers	Read	Employee Requests (all items), approve through the flow
Portal HR Admins	Full Control	All portal lists and libraries
Portal Auditors	Read	All lists and libraries

Solution steps

```
// Reuses site, vhdr and getDigest() from Case 2. Run as a site collection admin:
// breaking inheritance without copying removes inherited access.
```

```
async function post(url, body, method) {
  var h = { ...vhdr, 'X-RequestDigest': await getDigest() };
  if (method) { h['X-HTTP-Method'] = method; h['IF-MATCH'] = '*'; }
  var r = await fetch(site + url, { method: 'POST', headers: h, body: body ? JSON.stringify(body) : undefined });
  console.log(url, r.status);
  var t = await r.text();
  return t ? JSON.parse(t) : null;
}
async function getId(url) {
  var r = await fetch(site + url, { headers: vhdr });
  return (await r.json()).d.Id;
}

// 1. Create the four groups
for (var g of ['Portal Employees', 'Portal Managers', 'Portal HR Admins', 'Portal Auditors']) {
  await post('/_api/web/sitegroups', { __metadata: { type: 'SP.Group' }, Title: g });
}

// 2. Helper: give a group a permission level on a list
async function grant(list, group, role) {
  var gid = await getId(`/_api/web/sitegroups/getbyname('${group}')?$select=Id`);
  var rid = await getId(`/_api/web/roledefinitions/getbyname('${role}')?$select=Id`);
  await
post(`/_api/web/lists/getbytitle('${list}')/roleassignments/addroleassignment(principalid=${gid},roledefid=${rid})`);
}

// 3. Employee Requests: unique permissions + own-items-only security
var req = `/_api/web/lists/getbytitle('Employee Requests')`;
await post(`${req}/breakroleinheritance(copyRoleAssignments=false,clearSubscopes=true)`);
await post(req, { __metadata: { type: 'SP.List' }, ReadSecurity: 2, WriteSecurity: 2 }, 'MERGE');
await grant('Employee Requests', 'Portal HR Admins', 'Full Control');
await grant('Employee Requests', 'Portal Employees', 'Contribute');
await grant('Employee Requests', 'Portal Auditors', 'Read');

// 4. Employee Documents
await post(`/_api/web/lists/getbytitle('Employee
Documents')/breakroleinheritance(copyRoleAssignments=false,clearSubscopes=true)`);
await grant('Employee Documents', 'Portal HR Admins', 'Full Control');
await grant('Employee Documents', 'Portal Employees', 'Contribute');

// 5. Add a user to a group
await post(`/_api/web/sitegroups/getbyname('Portal Employees')/users`,
  { __metadata: { type: 'SP.User' }, LoginName: 'i:0#.f|membership|user@yourtenant.onmicrosoft.com' });
```

`ReadSecurity 2` and `WriteSecurity 2` mean users read and edit only the items they created. Managers who must see all requests are added to `Portal HR Admins`, or you grant them access through the approval flow (Case 10). Prefer groups over individual permissions so access reviews stay simple.

Expected result: a test employee sees only their own requests, and an HR admin sees all of them.

Case 4: Custom SPFx React dashboard

Scenario: give users a single landing page with request statistics and recent activity.

Solution steps

1. Create the folder structure: `components/Dashboard.tsx`, `components/RequestGrid.tsx`, `services/RequestService.ts`, `models/IRequest.ts`.
2. Define the model:

```
export interface IRequest {
  Id: number; Title: string; RequestType: string; Priority: string;
  Status: 'Pending' | 'Approved' | 'Rejected'; Created: string;
  Author?: { Title: string };
}
```

3. Build the dashboard with summary cards (Total, Pending, Approved, Rejected):

```
const Dashboard: React.FC<{ svc: RequestService }> = ({ svc }) => {
  const [items, setItems] = React.useState<IRequest[]>([]);
  React.useEffect(() => { svc.getAll().then(setItems).catch(console.error); }, []);
  const count = (s: string) => items.filter(i => i.Status === s).length;
  return (
    <div>
      <Stack horizontal tokens={{ childrenGap: 16 }}>
        <StatCard label='Total' value={items.length} />
        <StatCard label='Pending' value={count('Pending')} />
        <StatCard label='Approved' value={count('Approved')} />
        <StatCard label='Rejected' value={count('Rejected')} />
      </Stack>
      <RequestGrid items={items} />
    </div>
  );
};
```

4. Use Fluent UI (`DetailsList`, `Stack`, `Persona`) for layout and make the web part responsive.

Expected result: the dashboard shows live counts and the latest requests.

Case 5: CRUD operations with PnPjs

Scenario: the UI needs to create, read, update and delete requests without custom REST plumbing.

Solution steps

1. Configure PnPjs once, passing the web part context from `onInit`:

```
import { spfi, SPFx, SPFI } from '@pnp/sp';
import '@pnp/sp/webs'; import '@pnp/sp/lists'; import '@pnp/sp/items';
import { WebPartContext } from '@microsoft/sp-webpart-base';

let _sp: SPFI;
export const getSP = (ctx?: WebPartContext): SPFI => {
  if (ctx) { _sp = spfi().using(SPFx(ctx)); }
  return _sp;
};
```

2. Create the service layer:

```
export class RequestService {
  private list = () => getSP().web.lists.getByTitle('Employee Requests');

  getAll(): Promise<IRequest[]> {
    return this.list().items
      .select('Id','Title','RequestType','Priority','Status','Created','Author/Title')
```

```
    .expand('Author').orderBy('Created', false).top(100)();
 }
 create(r: { Title: string; RequestType: string; Priority: string; Description: string }) {
  return this.list().items.add(r);
 }
 update(id: number, changes: Partial<IRequest>) {
  return this.list().items.getById(id).update(changes);
 }
 remove(id: number) {
  return this.list().items.getById(id).recycle(); // Recycle Bin, not permanent delete
 }
}
```

3. Wire a form (Fluent UI `Panel`) to `create` and `update`, and a confirm dialog to `remove`.

Best practice: use `recycle()` instead of `delete()` so removed items can be restored, and always `select` only the columns you need.

Case 6: Search and filter

Scenario: users need to find requests and documents quickly.

Solution steps

1. Add a `SearchBox` and dropdowns for Status and Request Type, then build an OData filter:

```
async search(text: string, status?: string, type?: string): Promise<IRequest[]> {
 const f: string[] = [];
 if (text)   f.push(`substringof('${text.replace(/'/g, `''`)}',Title)`);
 if (status) f.push(`Status eq '${status}'`);
 if (type)   f.push(`RequestType eq '${type}'`);
 let q = this.list().items.select('Id','Title','RequestType','Status','Created');
 if (f.length) q = q.filter(f.join(' and '));
 return q.orderBy('Created', false).top(100)();
}
```

2. Debounce the search box (300 ms) to avoid a request per keystroke.
3. For cross-library full-text search, use SharePoint Search:

```
import '@pnp/sp/search';
const r = await getSP().search({ Querytext: `${text} path:"${siteUrl}"`, RowLimit: 20,
 SelectProperties: ['Title','Path','Author','LastModifiedTime'] });
```

Expected result: typing a name, choosing a status, or both, narrows the grid instantly.

Case 7: Microsoft Graph integration

Scenario: show organisation data (profile, manager, photo) without storing it in SharePoint.

Solution steps

1. Request permissions in `config/package-solution.json`:

```
"webApiPermissionRequests": [
 { "resource": "Microsoft Graph", "scope": "User.Read" },
 { "resource": "Microsoft Graph", "scope": "User.ReadBasic.All" }
]
```

2. After deployment, a tenant admin approves them in the SharePoint admin center under **Advanced > API access**.

3. Call Graph with the SPFx client:

```
const client = await this.context.msGraphClientFactory.getClient('3');
const me = await client.api('/me')
  .select('displayName,jobTitle,department,mail,officeLocation').get();
const manager = await client.api('/me/manager').select('displayName,mail').get();
```

4. Handle a `404` for users without a manager or photo with a `try/catch` and show a friendly fallback.

Test first in Graph Explorer with `GET https://graph.microsoft.com/v1.0/me/manager`, then move the call into code.

Case 8: User and profile information

Scenario: the dashboard header shows who is signed in, their manager, and what role they have.

Solution steps

1. Load the photo and build a `Persona`:

```
const blob = await client.api('/me/photo/$value').responseType('blob').get();
const photoUrl = URL.createObjectURL(blob);
```

2. Read SharePoint profile properties and site groups with PnPjs:

```
import '@pnp/sp/profiles'; import '@pnp/sp/site-users/web';
const props = await getSP().profiles.myProperties();
const groups = await getSP().web.currentUser.groups();
const isHR = groups.some(g => g.Title === 'Portal HR Admins');
```

3. Use `isHR` to show an **Admin** tab (all requests, user management) only to HR. Remember that hiding UI is a convenience; real security comes from the permissions in Case 3.

Expected result: the header shows name, job title, manager and photo, and HR users see the extra tab.

Case 9: Employee document management

Scenario: employees upload documents with metadata, and HR can find them by type or expiry date.

Solution steps

1. Upload a file and set metadata in one flow:

```
import '@pnp/sp/files'; import '@pnp/sp/folders';
const res = await getSP().web.getFolderByServerRelativePath('Employee Documents')
  .files.addUsingPath(file.name, file, { Overwrite: true });
const item = await res.file.getItem();
await item.update({ DocumentType: 'ID Proof', EmployeeId: currentUserId });
```

2. List documents with their size and editor:

```
const docs = await getSP().web.lists.getByTitle('Employee Documents').items
  .select('Id','FileLeafRef','DocumentType','Modified','Editor/Title','File/Length')
  .expand('Editor','File').filter('FSObjType eq 0').orderBy('Modified', false).top(50)();
```

3. Enable versioning and a retention label on the library for audit and recovery.

Expected result: uploaded files appear with the right type and owner, and older versions are retained.

Case 10: Power Automate approval workflow

Scenario: every new request must be approved by the employee's manager, and the outcome must be written back.

Solution steps

1. Create an **Automated cloud flow** with the trigger **When an item is created** (site `your portal site`, list `Employee Requests`).
2. Add **Get manager (V2)** (Office 365 Users) using the item's **Created By Email**.
3. Add **Start and wait for an approval**, type *Approve/Reject - First to respond*, assigned to the manager's mail. Use the item title, type, priority and description in the details.
4. Add a **Condition** on `Outcome` is equal to `Approve`.
5. In both branches use **Update item** to set `Status` (Approved or Rejected) and `ApproverComments` from the approval response.
6. Send an **email** to the requester with the result.
7. Turn on a **Configure run after** branch for failures that notifies HR Admins.

Use a service account or connection reference as the flow owner so it keeps running when staff change. Because the flow edits items that employees cannot edit, it needs a connection with edit rights on the list.

Expected result: creating a request sends an approval card in Outlook or Teams, and the item status changes automatically.

Case 11: Custom SPFx extensions

Scenario: show a policy banner on every portal page and colour-code the Status column.

Solution steps

1. Add an Application Customizer to the same solution:

```
yo @microsoft/sharepoint
```

Choose **Extension (Preview) > Application Customizer**, name it `PortalBanner`.

2. Render into the Top placeholder:

```
private _top?: PlaceholderContent;

public onInit(): Promise<void> {
  this.context.placeholderProvider.changedEvent.add(this, this._render);
  this._render();
  return Promise.resolve();
}
private _render(): void {
  if (!this._top) {
    this._top = this.context.placeholderProvider.tryCreateContent(PlaceholderName.Top);
  }
  if (this._top?.domElement) {
    this._top.domElement.innerHTML =
      `<div style='background:#0f6cbd;color:#fff;padding:8px 16px'>${this.properties.message}</div>`;
  }
}
```

3. Add a Field Customizer for `Status`:

```
public onRenderCell(event: IFieldCustomizerCellEventParameters): void {
  const colours: Record<string,string> = { Pending: '#c19c00', Approved: '#107c10', Rejected: '#d13438' };
  const v = event.fieldValue as string;
  event.domElement.innerHTML =
    `<span style='background:${colours[v] || '#666'};color:#fff;padding:2px 8px;border-radius:10px'>${v}</span>`;
}
```

4. Test with `?debugManifestsFile=` and `loadSPFX=true` query strings before packaging.

Expected result: the banner appears on all pages and the Status column shows coloured badges.

Case 12: Package, deploy and test

Scenario: release the solution to SharePoint Online and validate it end to end.

Solution steps

1. Build and package for production:

```
heft test --clean --production
heft package-solution --production
```

(On older SPFx versions use `gulp bundle --ship` and `gulp package-solution --ship`.) The package is created at `sharepoint/solution/employee-portal.sppkg`.

2. Upload employee-portal.sppkg to the App Catalog (Apps for SharePoint library), tick the tenant-wide option and choose Deploy. Then confirm with REST:

```
// Run from the console on any SharePoint page: confirm the package is deployed
var r = await
fetch(`${_spPageContextInfo.webServerRelativeUrl}/_api/web/tenantappcatalog/AvailableApps?$select=Title,ID,Deployed,CurrentVersionDeployed`,
  { headers: { Accept: 'application/json;odata=nometadata' } });
console.table((await r.json()).value);
```

3. Approve the Graph permissions under **SharePoint admin center > Advanced > API access**.
4. Add the web part to the portal home page. Register the banner on the site with REST (reuses the helpers from Case 2):

```
var r = await fetch(`${site}/_api/web/usercustomactions`, {
  method: 'POST',
  headers: { ...vhdr, 'X-RequestDigest': await getDigest() },
  body: JSON.stringify({
    __metadata: { type: 'SP.UserCustomAction' },
    Title: 'PortalBanner',
    Name: 'PortalBanner',
    Location: 'ClientSideExtension.ApplicationCustomizer',
    ClientSideComponentId: '<component-guid>',
    ClientSideComponentProperties: '{"message":"Welcome to the Employee Portal"}'
  })
});
console.log(r.status); // 201 = registered
```

5. Run the acceptance tests:

Test	Expected result
Employee creates a request	Item saved, approval email sent to manager

Test	Expected result
Manager approves	Status becomes Approved, employee is notified
Employee opens requests	Sees only own items
HR Admin opens dashboard	Sees all items and the Admin tab
Search by title and status	Grid filters correctly
Upload a document	Appears with metadata and version 1.0
Delete a request	Moves to Recycle Bin and can be restored
Graph profile card	Name, title, manager and photo load

4. Conclusion

The Employee Document & Request Management Portal replaces scattered email and shared-drive processes with one governed solution. It combines SharePoint lists and libraries for data, PnPjs for clean CRUD, Microsoft Graph for organisational context, Power Automate for approvals, and SharePoint groups for role-based security. SPFx web parts and extensions deliver a modern, consistent user experience, and the whole solution ships as a single `.sppkg` package.

The project demonstrates the full SharePoint developer lifecycle: design, build, secure, automate, package and deploy. Natural next steps are Teams integration, a mobile-friendly view, Power BI reporting on request turnaround times, and an automated CI/CD pipeline for releases.

Deliverables checklist

- Source code repository for `employee-portal`.
- REST provisioning scripts (Cases 2 and 3).
- Power Automate flow export (Case 10).
- `employee-portal.sppkg` and deployment notes (Case 12).
- Test evidence and screenshots for the acceptance table.